

Final Year Project — Full Build Order

Research Agent System • Project Development Plan

★ = Unique / proposed module

Phase 1 — Foundation

1. Document ingestion + chunking pipeline.
2. Embeddings + ChromaDB vector store.
3. Web search integration using Tavily.
4. **Guardrail — Input Sanitizer**: strips suspicious/injected instructions from scraped web content before it reaches an agent.

Phase 2 — Core Agents

5. **Researcher Agent**: pulls information from the vector DB and web search.
6. **Fact-Checker Agent**: performs cross-source verification and assigns a confidence score to each claim.
7. **Summarizer Agent**: condenses verified research notes.
8. **Writer Agent**: compiles the final cited report.
9. Wire all four agents into LangGraph with the existing fact-check → re-research loop.

Safe Minimum Viable Project

Milestone: Phase 1 + Phase 2 form the safe minimum viable project. If no ★ features are completed, this baseline is still a complete working research-agent system.

Phase 3 — ★ Planning-First Layer

10. Add a **Planning Agent** before the Orchestrator. It generates a sub-question breakdown and presents the plan to the user.
11. Add an **approval checkpoint** using LangGraph's *interrupt_before*, allowing the user to edit or approve the plan before execution.
12. **Why this order**: the Orchestrator must already be working before inserting a planning step in front of it.

Phase 4 — ★ Adaptive Self-Correction

13. Extend the Fact-Checker's confidence score into a **retry trigger**. If confidence is below a threshold, log the reason for failure instead of blindly looping.
14. Build the **Adaptive Retry Module**. It uses the failure reason to choose a different strategy, such as switching from web search to the vector DB, broadening the query, or searching academic sources only.
15. **Cap retries** at approximately 2–3 attempts to prevent infinite loops and unnecessary API usage. This cap is an important design decision to explain in the final report.

Phase 5 — ★ Contradiction Checker

16. Add a node between the Fact-Checker and Summarizer that compares outputs **across agents**, rather than only across sources.
17. Flag contradictions before they reach the Summarizer, preventing conflicting or incorrect information from being silently smoothed over during final writing.

Phase 6 — ★ Groundedness / Hallucination Red-Flag

18. After the Writer produces the final report, run a **verification pass**. For every claim and citation, check that it traces back to a real retrieved source.

19. If a claim cannot be matched to a source, mark it with a visible **■ warning** in the final output rather than silently retaining it.

20. **Priority feature**: this is the single most important ★ feature for demonstrating why the system produces trustworthy research. Prioritize it if development time becomes limited.

Phase 7 — ★ Prompt Optimizer

21. Build a lightweight preprocessing function that runs before every LLM call. It strips redundant context and compresses repeated instructions.

22. Track token usage before and after optimization to obtain a measurable result for the report, such as the percentage reduction in average token usage.

Phase 8 — Product Layer

23. **FastAPI backend + WebSocket streaming** for a live execution trace.

24. **Authentication + rate limiting**.

25. **React/Next.js frontend** displaying the live trace, plan-approval step, contradiction flags, and hallucination warnings.

Recommended implementation priority: Build the Foundation and Core Agents first → Planning-First Layer → Adaptive Self-Correction → Contradiction Checker → Groundedness/Hallucination Verification → Prompt Optimizer → Product Layer.